

boards to understand if the new feature or improvement has impacted core metrics.
- Consider running a Category Maturity Scorecard evaluation. | Product Manager |
|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Benchmark the experience: | Consider performing a UX Scorecard to score and benchmark the usability of the experience in the product based on a core set of heuristics and create new issues when identifying opportunities for improvement. | Product Designer |
|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Take action on Learnings: After you understand the qualitative and quantitative impact, you can take action on your learnings by creating new issues or updating existing open issues with more information. | - Open new issues or revise existing open issues for follow-on iterations and improvements.
- Ensure you've captured feedback in issues or as updates to your direction pages.
- If applicable, update your category maturity score and timeline.
- Share learnings with your group and stage.
- Consider sharing learnings with the broader team.
- Coordinate with your PMM to understand if there are any relevant Go-To-Market motions you should consider updating.
- Update experiment follow-up issue with results and specific next steps.
- Potentially create issues or MRs for updates to the documentation site, to provide useful information in advance of potential product updates related to learnings. | Product Manager |

Contributing to this page

All merge requests to this page require informing Product Operations prior to merging. To make updates such as grammatical fixes and typos, you can create an MR and assign Product Operations DRI @justinfarris as Reviewer for reference, and then merge. There's no need to wait for feedback on these types of updates prior to merging.

For updates that affect the overall phases by modifying core definitions, workflow labels or other cross-functionally utilized processes, you can create an issue or MR, add the current milestone and labels product development flow, product operations, prodops:release and assign it to Product Operations DRI@justinfarris as Reviewer for collaboration and approval.

The Product Operations DRI will ensure the MR gets included in product development flow release updates as well as ensuring alignment happens with the following stakeholders if needed:

- 1. CPO
- 1. VP of UX
- 1. VP of Development
- 1. VP of Quality Engineering
- 1. Product Leadership Team

In some cases, when significant changes to the product development flow are needed, affecting multiple teams and/or function workflows, Product Operations may request a draft page and/or a working group for multiple iterations prior to pushing changes to this framework, to minimize disruption and distraction during product development.

SECTION: product-development/product-development-flow/phase-template.md

Track Phase X: Phase Name

Label: ~label

Key Participants

<!-- this table outlines the individuals contributing (or informed) for this phase. DRI - Directly Responsible Individual, Collaborators - Active participation in the phase, Informed - kept up to date on the outcomes -->

Role	Function
DRI	TBD
Collaborators	TBD
Informed	TBD

Description

Phase description...

<!-- The description should capture the intent of the phase. Why does it exist and what are the basic motions involved with the phase? Be clear and write with brevity.

example from build > launch:

"After launch, the Product Manager and Product Designer should pay close attention to product usage data. This starts by ensuring your AMAU is instrumented and reporting as you expect. From there consider how the feature has impacted GMAU and SMAU. At this point you should also solicit customer feedback to guide follow-on iterative improvements, until success metrics are achieved/exceeded and a decision can be made that the product experience is sufficient. To create a combined and ongoing quantitative and qualitative feedback loop, the following activities are recommended:"

-->

Outcomes and Activities

<!-- The outcomes and related activities table below should capture the outcomes (recommended, or required) and the associated activities team members should consider deploying to achieve that outcome. Start by defining the outcomes, and layer in the activities where appropriate. If an outcome is required designate it in bold and use the 🚫 emoji.

example:

outcomes / outputs = What must be accomplished during the phase

activities = opportunities and suggestions to satisfy the outcomes / outputs

DRI = who is responsible for ensuring this outcome is completed (Product Manager, Engineering, Product Designer, SET, etc...)

-->

Outcomes	Activities	DRI
Outcome 1	- Activity 1	- Activity 2
Outcome 2	- Activity 1	- Activity 2

Required Outcomes

<kbd class="required">Required ·</kbd>

Outcome - Description of required outcome providing a better understanding of the outcome and any required outputs

SECTION: product-development/product-development-flow/product-development-budgeting.md

Pre-planning for potential new investments

The pre-planning is done in advance of the budget becoming available so that the potential investments are well documented and collaboration has occurred on them with the stakeholders.

| DRI | Task |

| --- | --- |

| Product Managers | Product managers do their planning for future potential investments in advance. This will be a list of ready-to-go business cases in advance, via creating issues using the potential investment issue template.. When crafting these proposals consider various gearing ratios including UX designer, UX Researcher, Tech Writer |

| Product Managers | Create private issues using the issue template and collaborate with PM and Engineering teams via tagging the appropriate development director, UX stable counterpart, SET stable counterpart, etc, as appropriate. |

| Section Leaders | Maintain a prioritized list of next investment asks vetted from proposal across their section |

Vetting and communication of new investment

| DRI | Task |

| --- | --- |

| Section Leaders | Make sure the funding plan encompasses all known things that need to be budgeted for successfully executing on the opportunity and ensure that the "investment" part of ROI calculation is reflective of the true investment across GitLab as a company. Prioritize investment asks within your Section. |

| VP of Product | Review plans and priorities, and ensure that product-wide priorities and global optimizations are accounted for. Provide a (private) video walkthrough of the approved hiring plan (as the spreadsheets tend to be hard to understand without an explanation). |

| CTO | Make sure the funding plan encompasses all known things that need to be budgeted from an Engineering perspective (such as not only new hires but comp reviews). |

| Product Managers | When a hiring plan is approved, document if previously approved reqs are or are not included. |

| Development, Quality, and UX VPs | Development, Quality, and UX VPs will work with their Directors to confirm and update budgeting to ensure it includes proper leadership ratios. |

Viewing existing potential investments

Search for issues with the label Investment Case

SECTION: product-development/product-development-flow/feature-flag-lifecycle/_index.md

NOTE:

The content below covers feature flags used by GitLab to deploy its own features, which is not the same as the feature flags offered as part of the product.

This page defines the process for the feature flags lifecycle at GitLab. Technical information for GitLab developers and code contributors can be found in the developer documentation.

When to use feature flags

The deployment of features behind feature flags is required for changes that could affect GitLab's availability of existing functionalities. If your changes only affect the new feature you're creating, use your judgment to determine if you need one or not.

Such changes include:

1. New features in high traffic areas (e.g., a new merge request widget, a new option in issues/epics, a new CI functionality).
1. Complex security or performance improvements that may require additional testing in production (e.g., rewriting complex queries, reusing complex finders in new ways, destructive database updates or changes to frequently used API endpoints).
1. Invasive changes to the user interface (e.g., introducing a new navigation bar, removal of a sidebar, significant UI changes in issues or MR interface).
1. Introducing dependencies on third-party services (e.g., adding support for importing projects).
1. Changes to features that can cause data corruption or cause data loss (e.g., features that process repository data or user-uploaded content).
1. Changes to permissions that will add restrictions (e.g., requiring Owner permissions instead of Developer)

Situations where you might consider not using a feature flag:

1. Adding a new API endpoint.
1. Introducing new features in low traffic areas (e.g., adding new export functionality in the admin area, group settings area, or project settings area).
1. Non-invasive frontend changes (e.g., slightly changing the color of a button, or moving a UI element in a low traffic area).

In all cases, if you're working on the changes like these, ask yourself:

> Why do I need to add a feature flag? If I don't add one, what options do I have to control the impact on application reliability and user experience?

For perspective on why we limit our use of feature flags please watch the video [Feature flags only when needed](#).

In case you are uncertain whether feature flag is necessary, please ask about this early in your merge request review process, and reviewers will likely provide you with an answer.

NOTE:

Feature flags are not supported in GitLab Dedicated.

The benefits of feature flags

<details>

It may seem like feature flags are configuration, which goes against our convention-over-configuration principle. However, configuration is by definition something that is user-manageable. Feature flags are not intended to be user-editable. Instead, they are intended as a tool for Engineers and Site Reliability Engineers to use to de-risk their changes. Feature flags are the shim that gets us to Continuous Delivery with our monorepo and without having to deploy the entire codebase on every change. Feature flags are created to ensure that we can safely rollout our work on our terms. If we use Feature Flags as a configuration, we are doing it wrong and are indeed in violation of our principles. If something needs to be configured, we should intentionally make it configuration from the first moment.

Some of the benefits of using development-type feature flags are:

1. It enables Continuous Delivery for GitLab.com.
1. It significantly reduces Mean-Time-To-Recovery.
1. It helps engineers to monitor and reduce the impact of their changes gradually, at any scale,
 - allowing us to be more metrics-driven and execute good DevOps practices, shifting some responsibility "left".
1. Controlled feature rollout timing: without feature flags, we would need to wait until a specific deployment was complete (which at GitLab could be at any time).
1. Increased psychological safety: when a feature flag is used, an engineer has the confidence that if anything goes wrong they can quickly disable the code and minimize the impact of a change that might be risky.
1. Improved throughput: when a change is less risky because a flag exists, theoretical tests about scalability can potentially become unnecessary or less important. This allows an engineer to potentially test a feature on a small project, monitor the impact, and proceed. The alternative would be to build complex benchmarks locally, or on staging, or on another GitLab deployment, which has a large impact on the time it can take to build and release a feature.

</details>

The cost of feature flags

<details>

When reading about the process, you can think that deploying a feature behind a feature flag

adds a lot of work. Fortunately, this is not the case, and we'll show you why. For this example, we'll specify the cost of the work to do as a number, ranging from 0 to infinity. The greater the number, the more expensive the work is. The cost does not translate to time, it's just a way of measuring complexity of one change relative to another.

Let's say we are building a new feature, and we have determined that the cost of this is 10. We have also determined that the cost of adding a feature flag check in a variety of places is 1. If we do not use feature flags, and our feature works as intended, our total cost is 10. This, however, is the best-case scenario. Optimizing for the best-case scenario is guaranteed to lead to trouble, whereas optimizing for the worst-case scenario is almost always better.

To illustrate this, let's say our feature causes an outage, and there's no immediate way to resolve it. This means we'd have to take the following steps to resolve the outage:

1. Revert the release.
1. Perform any cleanups that might be necessary, depending on the changes that were made.
1. Revert the commit, ensuring the main branch remains stable. This is especially necessary if solving the problem takes days or even weeks.
1. Pick the revert commit into the appropriate stable branches, ensuring we don't block any future releases until the problem is resolved.

As history has shown, these steps are time-consuming, complex, often involve many developers, and, the worst of all: our users will have a bad experience using GitLab.com until the problem is resolved.

Now let's say that all of this has an associated cost of 10. This means that, in the worst-case scenario, which we should optimize for, our total cost is now 20.

If we had used a feature flag, things would have been very different. We don't need to revert a release, and because feature flags are disabled by default, we don't need to revert and pick any Git commits. In fact, all we have to do is disable the feature, and in the worst case, perform a cleanup. Let's say that the cost of this is 2. In this case, our best case cost is 11: 10 to build the feature, and 1 to add the feature flag. The worst case cost is now 13:

- 10 to build the feature.
- 1 to add the feature flag.
- 2 to disable and clean up.

Here we can see that, in the best-case scenario, the work necessary to resolve the issue is only a tiny bit bigger when compared to not using a feature flag. Meanwhile, the process of reverting our changes was significantly and reliably cheaper.

In other words, feature flags do not slow down the development process. Instead, they speed up the process as managing incidents now becomes much easier. As

continuous deployments are easier to perform, the time to iterate on a feature is reduced even further, as you no longer need to wait weeks before your changes are available on GitLab.com.

</details>

How to use Feature Flags

For an overview about starting with feature flags in GitLab development, follow the training template.

Before using feature flags, make sure to read the information on this page and the following development guides:

1. Developing with feature flags: Learn about the types of feature flags, their definition and validation, how to create them, frontend and backend details, and other information.
1. Documenting features deployed behind feature flags:
How to document features deployed behind feature flags according to their state, and how to update the docs when their state changes.
1. Controlling feature flags: Learn the process for deploying a new feature, enabling it on GitLab.com, communicating the change, logging, and cleaning up.
1. Feature flags and the change management process: Learn whether your flag requires use of the change management process or not.
1. Checking on the status of your teams feature flags: See if your team has too many, or stale feature flags. See how your feature flags have been toggled over time.
1. Conducting an experiment with feature flags: Learn how the Growth department uses experiment feature flags.

For information about how the user will interact with features behind flags, see:

1. How GitLab administrators can enable and disable features behind flags:
An explanation for GitLab administrators about how they can enable or disable GitLab features behind feature flags.
1. What "features deployed behind flags" means to the GitLab user:
An explanation for GitLab users regarding how certain features might not be available to them until they are enabled.

Feature Flag Lifecycle

Planning

1. Evaluate if you need a feature flag:
 1. Process for evaluation.
 1. You, as an engineer, come up with a proposed solution to the issue you're working on and decide whether you'll implement it behind a feature flag.
1. Decide how you will implement the feature flag and its rollout according to:
 1. Choose the feature flag type.

1. Decide on the definition and plan out the YAML.
1. Decide on what type of actor to bind your feature flag to (if any).
1. Think about where your feature flag should toggle in the code.
1. Create a feature flag definition in code and make a follow-up rollout issue:
 1. Create a feature flag.
 1. Create a feature flag rollout issue using the template (if necessary).
 1. Ping the EM/PM on the feature flag rollout issue to get it scheduled/planned/refined.
 1. EM and Engineer work together to finalize the rollout plan for the feature flag. Not all steps in the feature flag rollout template are mandatory for every feature flag.

Development

1. Add the feature flag into the code for the backend, frontend, and in tests:
 1. Develop with the feature flag in your vertical slice.
 1. Include the feature flag in tests to check its behavior when enabled and disabled.
 1. Flip the feature flag locally to ensure it works.
1. Add the feature behind the flag to the codebase through an MR following the implementation process.
1. Continue to iterate towards issue completion, using the feature flag to test logical slices:
 1. You may use your feature flag in more than one MR. Continue to iterate until the issue is complete.
1. Ensure you follow the documentation guidelines for feature flags and keep them up-to-date according to the feature flag state.
1. When slices are deemed complete enough (keeping in mind low level of shame), proceed to the rollout process.
 1. Some teams may choose to close the feature issue here once it is complete, other teams may want to wait until after the rollout process is finished. If you close your feature issues after the code is present in the default branch, you should close the issue here.

Rollout

1. Finalize the rollout plan:
 1. Following the rollout guidelines determine a plan for rolling out your feature flag.
1. Begin rollout plan:
 1. Rollout plans vary flag to flag, take the steps you've outlined in your feature flag rollout issue.
1. Removing the flag, cleaning up, and feature announcement:
 1. Follow the feature flag clean up process.
 1. Ensure that the feature flag documentation process has been followed.
 1. Ensure the flag is removed from the code and from the database with the chatops feature flag removal command.
 1. If your team closes the feature issue after the feature flag has been removed, it should be closed here.
 1. If the originating issue will be included in the Release Post coordinate with Engineering Manager, Product Manager, Product Designer, Product Marketing Manager, and Technical writer to include the Release Post Item

Flowchart

mermaid

```

graph TD
    A[Start] --> B{Evaluate need}
    B -->|No| E[End]
    B -->|Yes| C[Decide on implementation]
    C --> D[Create flag / create rollout issue]
    D --> F[Add flag and first iteration into code]
    F --> G[MR into codebase]
    G --> H[Iterate on issue]
    H --> I{Feature issue complete?}
    I -->|No| H
    I -->|Yes| J[Finalize rollout plan]
    J --> K[Begin rollout]
    K --> L[Complete rollout]
    L --> M[Remove flag]
    M --> E
    M --> O[Close rollout issue]
    O --> N[Close feature issue]
    N --> O

```

A --> B;
 B ----> |No| E;
 B --> |Yes| C;
 C --> D;
 D --> F;
 F --> G;
 G --> H;
 H --> I;
 I --> |No| H
 I --> |Yes| J;
 I -. Optional Yes .-> N;
 J --> K;
 K --> L;
 L --> M;
 M --> E;
 M --> O;
 M -. Optional Yes .-> N

Including a feature behind feature flag in the final release

In order to build a final release and present the feature for self-managed users, the feature flag should be at least defaulted to on. If the feature is deemed stable and there is confidence that removing the feature flag is safe, consider removing the feature flag altogether. It's strongly recommended that the feature flag is enabled globally on production for at least one day before making this decision. Unexpected bugs are sometimes discovered during this period.

The process for enabling features that are disabled by default can take 5-6 days from when the merge request is first reviewed to when the change is deployed to GitLab.com. However, it is recommended to allow 10-14 days for this activity to account for unforeseen problems.

Feature flags must be documented according to their state (enabled/disabled),

and when the state changes, docs must be updated accordingly.

Changing the default state or removing the feature flag has to be done before the release date, at least 3-4 working days before, in order for the change to be included in the final self-managed release.

Note: consider that, by removing the flag, the feature will become available on GitLab.com shortly after the merge request was merged.

In addition to this, the feature behind feature flag should:

- Run in all GitLab.com environments for a sufficient period of time. This time period depends on the feature behind the feature flag, but as a general rule of thumb 2-4 working days should be sufficient to gather enough feedback.
- The feature should be exposed to all users within the GitLab.com plan during the above mentioned period of time. Exposing the feature to a smaller percentage or only a group of users might not expose a sufficient amount of information to aid in making a decision on feature stability.

While rare, release managers may decide to reject picking or revert a change in a stable branch, even when feature flags are used. This might be necessary if the changes are deemed problematic, too invasive, or there simply isn't enough time to properly measure how the changes behave on GitLab.com.

Community contributions

Feature flag rollouts require access to internal GitLab systems. Because of this restriction, a community contribution will require assistance from a GitLab team member to rollout the feature flag.

During the planning phase it is essential that the EM and PM are pinged by the MR Coach or a GitLab Team member, who executes the review, on the feature flag rollout issue to get it scheduled/planned/refined. Also they should tag the feature flag rollout issue in the final MR using the feature flag, so that the EM and PM are aware that the rollout will be required soon.

Dashboard & Metrics

For more information on the lifecycle of a feature flag, such as how many are introduced per release, how long they exist in the application, or how long they've been enabled, see this dashboard (internal only). We also carry 7 days worth of data in Kibana (internal only) and a GitLab project feature-flag-log holds the same indefinitely for feature flags that become disabled or enabled.

SECTION: product-development/product-development-flow/success-metrics/_index.md

Quarterly Survey Results

Product Development Flow Survey Q2 2020

Cycle Time Metrics in Sisense

Dashboard

Q4 2020 Before vs. After Comparison

An initial quantitative analysis was done for the first major update of the PDF in December 2020 looking at:

- Are we driving more opportunities for collaboration?
- Are we reducing the total steps required?
- Have we simplified things (judged by word count)

For further details visit Issue 9243

	Collaboration Touchpoints	Total Steps	Word Count
Old Flow	11	68	6025
New Flow	37	30	4934

SECTION: product-development/r-and-d-interlock/_index.md

The R&D Interlock Process is used to align Product Management, User Experience, and Engineering teams on roadmap planning and coordinating execution. The interlock consists of three major components:

- The Resource Allocation Framework provides a structure to map product and engineering driven efforts to the available engineering resources.
- The Roadmap Structure describes the planning artifacts (roadmaps) used for alignment and defines their ownership, purpose and audience.
- The R&D Alignment Process provides a process and timeline to create alignment for the next execution quarters.

The alignment process outlines a joint approach across product-driven and engineering-driven initiatives while ensuring clear communication channels with various stakeholders.

Resource Allocation Framework

We're establishing a balanced approach between product-led and engineering-driven initiatives that prioritizes customer needs, quality standards, and long-term product sustainability. This balance serves as the foundation for ongoing dialogue between Product Management and Engineering teams, with the flexibility to adjust ratios as needed for each team's specific context.

Every initiative is going to be tracked as a separate slide in the R&D Execution Roadmap and (after alignment) represented in the R&D Investment Roadmap, the Internal Roadmap and the Public Roadmap.

- P1/P2/P3: Product-Driven Initiatives
 - Prioritization Levels
 - P1: 100% Eng Confidence that the complete initiative will be delivered at the committed date
 - P2: 80% Eng Confidence that the complete initiative will be delivered at the committed date
 - P3: 50% Eng Confidence, can be stopped if a P1-2 or E1-2 is at risk.
 - Some but not all P1/P2/P3 projects will be added to the Public Roadmap (owned by GTM) and will be labeled with the GTM tiers T1/T2/T3 (reference: definition of GTM tiers).
 - It's possible that T and P priorities do not strictly match. For example, a contractual customer commitment might be a P1 for Product but a T3 for GTM. Similarly, a competitive gap might be a P2 for Product but a T3 for GTM. It is expected that this divergence of priorities will happen frequently.
 - GTM priorities T1-3 can never be higher (only ever be equal or lower) than Product priorities P1-P3, else we'll run into priority inversion. Exceptions need PLT/ELT/MLT approval.
 - T priorities are externally communicated to stakeholders via Public Roadmap
 - P priorities are externally communicated to stakeholders via

Internal Roadmap

- E1/E2/E3: Engineering-Driven Initiatives

- Prioritization Levels:

- E1: 100% Eng Confidence that the complete initiative will be delivered at the committed date
- E2: 80% Eng Confidence that the complete initiative will be delivered at the committed date
- E3: 50% Eng Confidence, can be stopped if a P1-2 or E1-2 is at risk.
- Internal visibility only
- Not externally communicated

Unless approved by the respective VP of Product and/or Engineering, there is a maximum limit of 1 x (P1 or E1) and 2 x (P2 or E2) per 20 engineers with a minimum of 1 x P2 per key investment areas (e.g. SCM, CI, Security, Compliance, Planning, Duo).

Creating a clean alignment ("interlock") between Product and Engineering requires a structured planning methodology that integrates user requirements with focused quality improvements and essential technical efforts. This framework establishes a dual-track system where product-driven initiatives (P1/P2/P3) operate with explicit resource allocation parameters: P1 receiving engineering capacity to deliver 100% of planned scope with 100% certainty and full visibility in external communications, P2 allocated engineering resources to deliver 100% of planned scope with 80% certainty, both with defined acceptance criteria. P3 efforts are implemented through iterative development cycles when capacity permits with a target of 50% confidence. This systematic approach ensures proper resource allocation while creating a traceable relationship between committed functionality and execution.

Parallel to these requirement-driven developments, the framework implements a technical sustainability track (E1/E2/E3) with equivalent resource allocation metrics but isolated from external dependencies and release communications. This structure enables critical refactoring, dependency upgrades, test automation improvements, and infrastructure optimization to proceed with appropriate prioritization without external scheduling constraints.

The configurable ratio between P-track and E-track allocations provides implementation flexibility across different system components and architectural layers, allowing teams to adapt the framework based on technical debt accumulation, system stability metrics, and component lifecycle phases ultimately producing systems that satisfy functional requirements while maintaining architectural integrity.

Roadmap Structure

R&D Execution Roadmap

- Content: All P1/P2/P3 and E1/E2/E3 initiatives

- Cadence: Quarterly updates, 4-quarter rolling window
- Audience: Product and Engineering leadership
- Owner: Eng VP in alignment with Product
- Purpose: Readiness, Feasibility, and Execution Planning
- Format: One deck per section (template),
one roadmap overview for customer and engineering driven initiatives per stage, Individual alignment slides for each initiative

R&D Investment Roadmap

- Content: P1/P2 and E1/E2 initiatives (80%+ confidence)
- Cadence: Quarterly updates, 4-quarter rolling window
- Audience: Board level
- Owner: CPO and CTO Purpose: Strategic investment overview

Internal Roadmap

- Content: All P1/P2/P3 features
- Cadence: Quarterly updates, 4-quarter rolling window
- Audience: Select customers
- Owner: PM VP in alignment with Eng and UX
- Purpose: Product evolution and feature planning

Public Roadmap

- Content: A subset of P1/P2/P3 product-driven features will be labeled as T1/T2/T3 for GTM. N.b. that GTM priority T1/T2 can never be higher than product priority to avoid priority inversion.
- Cadence: Quarterly updates, 4-quarter rolling window
- Audience: Customers
- Owner: GTM in alignment with Product
- Purpose: External commitment tracking

mermaid

flowchart LR

```

ExecutionRoadmap("
  · R&D Execution Roadmap<br>
  Purpose: Readiness, Feasibility and Execution Planning<br>
  P1/P2/P3 and E1/E2/E3 Initiatives<br>
  Audience: PM, UX and Engineering Leadership
  Owner: Env VP in alignment with PM and UX
")

```

```

ProductRoadmap("
  · Product Roadmap<br>
  Purpose: Product Evolution and Features<br>
  P1/P2/P3 Initiatives<br>
  Audience: Select Customers
")

```

Owner: PM VP in alignment with Eng and UX
")

InvestmentRoadmap("
· R&D Investment Roadmap

Purpose: Strategic investment overview

P1/P2 and E1/E2 Initiatives

Audience: Board level
Owner: CPO and CTO
")

PublicRoadmap("
· Public Roadmap

Purpose: Strategic investment overview

P1/P2/P3 Initiatives

Audience: Select Customers
Owner: PM VP in alignment with Eng and UX
")

ExecutionRoadmap --> ProductRoadmap
ExecutionRoadmap --> InvestmentRoadmap
ProductRoadmap --> PublicRoadmap

R&D Alignment Process

The R&D Alignment process produces a generally agreed on R&D Execution Roadmap (and all derivative roadmaps) one quarter ahead of the planning window. If we're planning for Q_n to Q_{n+3}, the planning quarter is prior to execution and marked as Q_{n-1} to provide a clear planning timeline. The alignment process runs in Q_{n-1} Week 7-12.

mermaid

gantt

dateFormat YYYY-MM-DD
axisFormat End of Week %W
tickInterval 1week

title Quarterly Planning Timeline

Feature Alignment : 2025-02-16, 7d
R&D Alignment : 7d
GTM Alignment : 7d
Commitment : 7d
Comms : 7d
Retrospective : 7d

<h3 id="feature-alignment">Feature Alignment</h3> <p>Timeline: Q_{n-1} Week 8</p>	- Goal: Create PM/EM alignment on full set of P1-3/E1-3 features - Content: - Discuss P1/P2/P3 and E1/E2/E3 initial draft ideas - Identify ownership, dependencies and conflicts - Initial assessment of: - Business value - Technical feasibility - Engineering bandwidth and timeline - Resource requirements and Dependencies - Strategic alignment - Customer Zero requirements and enablement - Improvements (Quality, stability, security etc.) should be mapped as either P or E driven initiatives - Artifact: Stage-level alignment slides in R&D Execution Roadmap - Owner: PLT/ELT - Participants: Group PM/PD/EM for each capability under consideration - Format: Sync or async, as preferred by group-level teams
<h3 id="rd-alignment-discussion">R&D Alignment Discussion</h3> <p>Timeline: Q_{n-1} Week 9</p>	

Goal: Align Product & Engineering leadership on:

Scope and importance of capability to customer, quality and needs

Ability to commit to proposed E/P priority ranking

Content:

Discussion to align on:

Customer Problem and Business Value

Proposed solution and Definition of Good: Clear success and landing criteria

Proposed priority

Resource requirements

Initial timeline estimates

Dependencies identification

Risk assessment

Topic Granularity: Minimum threshold of 1 FTE quarter

Smaller initiatives are aggregated as milestones into thematic features

Artifact:

R&D Execution Roadmap with one joint alignment slide per feature (<a

href="https://www.google.com/url?q=https://docs.google.com/presentation/d/1UTjvJVI544gj9cYrmKeW8KI8dtXBZ6jzxywOulxRHrl/edit"

>template)

Owner: PLT/ELT

Participants:

- E-track: EM responsible for capability - P-track: PM & EM responsible for capability	
<h3 id="gtm-alignment-discussion">GTM Alignment Discussion</h3> <p>Timeline: Q_{n-1} Week 10</p>	
- Goal: Align R&D and GTM leadership on: - Scope and importance of capability to GTM considerations - Ability to commit to proposed T priority ranking - Content: - Pre-read: R&D joint alignment slide in R&D Execution Roadmap - Discussion artifact: Proposed T priorities from GTM - Owner: MLT - Participants: ELT, PLT & MLT	
<h3 id="final-prioritization-and-commitment">Final Prioritization and Commitment</h3> <p>Timeline: Q_{n-1} Week 11</p>	
Goal: Document outcome of alignment discussions in R&D Execution Roadmap	

```

</li>
Content:
<ul>
  <li>Final T/P/E assignments</li>
  <li>Delivery timeline commitment</li>
  <li>Resource allocation confirmation</li>
  <li>Identification of dependencies</li>
  <li>Documentation of risks</li>
</ul>
</li>
<li>
Owner:
<ul>
  <li>E-track: EM responsible for capability, Eng VP signoff</li>
  <li>
P-track: PM & EM responsible for capability, PM and Eng VP signoff
  </li>
</ul>
</li>
<li>Format: Async</li>
</ul>
</td>
</tr>
<tr>
<td>
<h3 id="upstream-communication">Upstream Communication</h3>
<p>Timeline: Q<sub>n-1</sub> Week 12</p>
</td>
<td>
<ul>
<li>
Aligned P1-3 initiatives integrated into
<a href="internal-roadmap">Internal Roadmap</a>
</li>
<li>
P1/P2 and E1/E2 initiatives integrated into
<a href="rd-investment-roadmap">R&D Investment Roadmap</a>
</li>
<li>
Select P1/P2 initiatives integrated into
<a href="public-roadmap">Public Roadmap</a> as T1/T2, n.b. that GTM
priority T1/T2 can never be higher than product priority to avoid
priority inversion.
</li>
</ul>
</td>
</tr>
<tr>
<td>
<h3 id="retrospective">Retrospective</h3>

```

Timeline: Q_n Week 1
- Goal: Learn from prior planning cycle and identify any changes that need to be implemented going forward. - Content: Collect, discuss and incorporate feedback from prior planning iteration - Artifact: Q_{n-2} Planning Retrospective. - Owners: PLT/ELT - Participants: Planners at Section and Stage Level

Success Metrics

- Roadmap alignment with company goals
- Planning efficiency (time spent planning and aligning, effort sizing, customer / quality / sustainability balance)
- Planning and execution accuracy (Delivery against commitments in time)

Commitment Change Requests

Any major changes to the timeline, scope, cost, priority, quality or risk for a committed feature of priorities P1/E1 and P2/E2 should be raised to respective PLT and ELT members as well as CPO and CTO via a Commitment Change Request to get explicit approval and keep all stakeholders in the loop:

1. Project DRIs update the alignment slide in the R&D Execution Roadmap and post a proposed update to [r-and-d-roadmap-changes](#), @ing in the respective stakeholders. The post should follow a standardized format, so decision makers can quickly respond / act:

text

Proposing change to feature: \$FEATURE

Change Type: [select: Timeline / Scope / Cost / Priority / Quality / Risk]

What's changing: <Priority from XX to YY>, <Delivery Milestone from YY.Y to ZZ.Z>, .

Background: [2-3 sentences describing the decision]

Impact: [optional, further detail on impact to customers / cost / list of projects that are

dependent on this project + @ mentions of DRIs for those projects]

Proposed by: [Name of Product and Engineering DRI]

Approvers: [specific PLT, ELT members], CPO, CTO

<more narrative / details of change, motivation, impact>

2. Free form discussion on the thread, optional meeting

3. Approval to commit to change from CPO and CTO

Any change that constitutes a slip in deadline, a significant reduction in scope, or anything else that would make us miss customer expectations constitutes a major change. If in doubt, go through the change management process.

SECTION: product-development/roles-and-responsibilities/_index.md

Successful product development requires and demands a unified commitment to shared outcomes across team members. The entire team must collectively embrace responsibility for the delivery of positive user impact, aligned to our GitLab value of delivering Results for Customers.

It's a Team Effort

Responsibilities to Deliver High Quality Product

Quality is Everyone's Responsibility

Role vs. Responsibility

There is an important distinction between a role (such as Product Manager, Engineering Manager) and a Lead's responsibility area (such as Product, Technical, Delivery, Resource, UX). While roles such as 'Product Manager' and 'Engineering Manager' often align with specific leadership responsibilities, their areas of responsibility are not exclusive to their roles.

- Ex: while a team member with the 'Product Manager' role often fulfills the Product Lead responsibilities, they have other critical contributions beyond the demarcations of the Product Lead role. In some situations where a product is consumed internally and may not directly interface with end customers, the Product Lead role may be fulfilled by a Technical Individual Contributor (IC) or an Engineering Manager.

- Ex: a team member with an 'Engineering Manager' role often covers multiple responsibilities

simultaneously - an Engineering Manager may fulfill some or all of the Technical Lead, Delivery Lead, and Resource Lead roles based upon the needs of their specific team.

With the caveats above, the general pattern we see within GitLab product development is:

- Product Division usually covers the responsibilities of:
 - Product Lead - note: there can be exceptions here, especially in the infrastructure area.
 - UX Lead
 - Documentation Lead
- Engineering Division usually covers the responsibilities of:
 - Technical Lead
 - Delivery Lead
 - Resource Lead

Within GitLab product development teams, we believe that "we win or lose as a team" - wins are team wins, and losses are team losses. The whole team owns the outcomes, and responsibilities assigned to a subset of the team are intended to drive execution excellence, and not meant to install rigid boundaries. One individual can cover multiple areas of responsibility on behalf of a team:

- Ex: Technical Lead may serve as the Delivery Lead
- Ex: Delivery Lead may also serve as Resource Lead for the team
- Ex: Technical Lead may serve as the Product Lead and Documentation Lead for technical excellence, reliability, scalability, or sustainability efforts.

Lastly, the specific needs that a product development team has for a specific responsibility may differ based upon the type of project, maturity of project, maturity of technology area and technology stack, maturity of developers, etc. The key is ensuring that each product development team has agreement and interlock on ownership and accountability for each of these responsibilities, regardless of how they are distributed among team members and across roles.

Who, What, Why, How, and When

For Product Development, we are leveraging the general-purpose questioning method of "who", "what", "why", "how", and "when" and asking these questions to better understand the outcomes we are aiming for the team. Each of us plays a role in answering these questions on a continuous basis to achieve delivering a world class product to our customers. In the figure below, we walk through an illustrative example of how Product Development team members contribute and collaborate in shaping a new product feature:

Our objective is to preserve each team's freedom to leverage agile methods to develop and deliver customer value in our product, while introducing a synchronized planning process to ensure that we deliver with the predictability and quality that GitLab's customers demand and deserve.

Key Events & Activities Mapping into Responsibilities

Hierarchy of Work

·
Product Focus | ·
Technical focus | · | | | |
 | 18 | Interlock on updated R&D roadmap based upon actuals | · | · | · | · | · | · |
 | 19 | Engage with PMM and DevRel to ensure messaging is correct, updated, and support the
 creation of GTM materials as needed | ·
Product Focus | | | | | · |
 | 20 | Present the parts of the product and related roadmap to customers in support of sales
 opportunities or updates for existing customers | · | | | | |

SECTION: resellers/_index.md

Welcome to the GitLab Channel Handbook

We invite organizations interested in becoming a GitLab Channel Partner to APPLY NOW.

This is the primary Handbook page for prospective and existing GitLab Channel partners and is organized into the following sections: Welcome to the GitLab Channel Handbook, Channel Program Guide, Channel Partners: Working with GitLab, and Channel Partners: Developing your GitLab Practice. This services as an introduction to the GitLab Channel Program and provides links to individual sections with additional detailed information.

GitLab Channel Introduction

The GitLab Partner Program supports organizations ranging from value added resellers (VARs) to systems integrators (SIs) to managed service providers (MSPs) and beyond looking to resell or refer GitLab sales opportunities, or provide services for GitLab customers. In addition, the program empowers partners to capture value from and support the entire customer lifecycle with guidance and support for everything from pre-sales engagement through full customer adoption of GitLab.

Why Become a GitLab Partner?

The right product at the right time. In any given market there are only a few times when the stars align with a product that is creating and leading a category, a product that helps customers redefine what success can look like, and with a robust opportunity for an ecosystem of partners to help customers maximize their ROI in that product.

GitLab represents one of those opportunities. We are still in the early phases of capturing an estimated \$40 billion dollar addressable market^[1]. A market that's evolving from point solutions to a platform and where GitLab offers the most comprehensive offering with proven ROI that partners can monetize by accelerating customers' successful adoption of GitLab.

From a strategic perspective, the comprehensive GitLab DevSecOps platform can be a driving force in your application modernization, digital transformation, cloud adoption, and even your security & compliance practices.

How to Navigate the Channel Guide

If you have browsed the GitLab Handbook before or this is your first time, we want to make your experience navigating the Channel Partner Program Handbook as easy as possible.

These pages document our shared team philosophy and processes, while the individual sections contain more detailed information. If you ever need to find your way back to either page or section, you can locate navigation links at the top or bottom of each page or section, as well as helpful Outline links located on the right of each page.

How to Navigate: To find your way back to the 1st Page [CLICK HERE](#).

Welcome to the GitLab Channel Handbook, Channel Program Guide, Channel Partners: Working with GitLab, and Channel Partners: Developing your GitLab Practice.

| GitLab Channel Partner Program Sections

|

| -----

| -----

| -----

| ----- |

|

|

| Channel Program Guide

|

| In this section you will learn how the GitLab Channel Partner Program works, the value GitLab brings to partners, Program Benefits & requirements, and an introduction to GitLab Services program.

|

|

|

| Channel Partners: Working with GitLab

|

| In this section, you will be given a deep dive into working with and managing the Four main aspects of the GitLab: (1) Administration, (2) Sales, (3) Marketing, (4) Technical, and (5) Distributors.

|

|

|

| Channel Partners: Developing your GitLab Practice

|

| In this section, you will learn how the GitLab Partner Enablement Program is designed to provide our partners with similar training as GitLab employees. Specifically developing core training, accreditation, and certification for sales, solution architects (pre-sales engineers), and post-sales engineers and consultants.

|

|

|

| Channel Champions: Joining GitLab Partners Champions Program

|

| In this section, you will be about GitLab Champions Program, an initiative designed to recognize and empower GitLab Channels key members who demonstrate exceptional advocacy, expertise, and enthusiasm for GitLab. Learn about the program's purpose, benefits, and how to become a champion, as well as the significance of joining this dynamic community of passionate GitLab advocates. |

| Channel Partners: Technical Presales Enablement

| In this section, we review assets and enablement from the perspective of Channel Partners' Technical Presales constituencies.

| Partner Operations (GitLab Internal)

| For GitLab Team members only. In this section, GitLab Team members will find important information on navigating the Partner Program's internal processes.

| GitLab Alliance Program

| This link will allow you to explore the exceptional technology companies we proudly partner with.

Contact Us

All authorized GitLab resellers are invited to the GitLab resellers Slack channel. This Slack channel allows you to reach out to our sales and marketing team in a timely manner, as well as other resellers.

Additionally, you can email the GitLab Channel Team at partnersupport@gitlab.com.

SECTION: resellers/channel-tools-resources.md

We invite organizations interested in becoming a GitLab Channel Partner to **APPLY NOW**.

GitLab Partner Portal

Partners are given access to our Partner Portal when they are onboarded. This serves as the primary source of information and transactions for our Partners. The GitLab Partner Portal has the following capabilities:

- Partner registration and account management
- Deal registration and opportunity management
- Training accreditation and other learning
- Sales, services, and program and marketing resources
- GitLab news, communications and program announcements.

Visit the portal at partners.gitlab.com.

Please note: if no one from your organization logs in to use our Partner Portal, after 6 months, your portal account will be deactivated. If you need your account reactivated, please contact partnersupport@gitlab.com and they will reactivate you.

GitLab Landing Page Guidelines

GitLab Channel Partners need a landing page with information about GitLab as part of their organization website. Here is what we look for in a webpage:

- We prefer your landing page URL be created as yourcompanyurl.com/GitLab. If this is not possible, we ask you to set a redirect for that URL to the actual one.
- We encourage your landing page to be in the local language. There are plenty of resources on GitLab, many in additional languages, however you can translate any of our Instant Marketing campaign materials (found in the marketing section of the Partner Portal) onto your website.
- Your organization's GitLab webpage should use the Authorized Reseller logo (available on the Partner Portal) and link back to us at about.gitlab.com.
- Your webpage should include a "What Is GitLab?" paragraph which can be found in our press kit along with our logo. You can find additional information in our reseller marketing kit.
- To ensure you are consistently representing our product on your website, we ask that wherever you mention the GitLab product or features, there should be a link back to the corresponding item on about.gitlab.com.
- While not required, we encourage Partners to include a Free Trial button on your GitLab webpage. This Free Trial button is customized to include your unique ID and tags all leads to your organization. The 30-day Free Trial helps your customer try GitLab for one month free and gives you the opportunity to discuss how it's working for them. To learn more about this offering go to the Marketing Demand Generation section of the Partner Portal.

Marketing Resources

Channel Partners have several programs and tools available to help them market and sell GitLab. The following are a few of the programs offered; additional programs are added periodically so bookmark this page to ensure you always have the latest information.

GitLab Brand and Logo

Bookmark the GitLab Brand and Logo website to ensure you have access to these materials.

Authorized Reseller Logo

The GitLab Authorized Reseller logo allows your prospects and customers to know that GitLab is working with you.

- Use the GitLab Authorized Reseller logo on your materials where appropriate and in accordance with our brand and logo guidelines.
- The logos are available in the README file of the resellers project. Also, the logos can also

be downloaded from the Marketing collection of the Asset Library in the Partner Portal.

GitLab Public Relations Request

Any requests for public relations support must go through the CAM as well as the GitLab PR team. Please refer to the Corporate Communications handbook page for more information on the process.

Partner Instant Marketing Campaigns

We've developed a full suite of GitLab marketing assets for Partners, including ebooks, infographics, videos, digital ads, social media posts, and more. They can select what they need, customize the materials with a logo, and begin marketing. Social media materials are also available. Materials are available in 6 languages: English, French, German, Japanese, Korean and Spanish.

Start using these materials today by going to the Marketing Demand Generation section of the Partner portal.

Partner Microsite Program

Keeping your Partner websites up-to-date with the latest GitLab content is easy with our Partner microsite program. Partner's simply provide GitLab with their logo, give us an "Abo